

Use the Bench Calculator to Program Mathematics

This third article in the mathematical journey through open source takes you through the functional power of the bench calculator.



After going through basic programming on the bench calculator, it's time to explore its functional power. As mentioned earlier, you can do functions with the bench calculator (BC). Unlike C, it has built-in functions, where as standard math functions, and user-defined functions are similar to C.

Built-in functions

The BC's built-in functions are:

- `length(expr)` – returns the number of significant digits in `expr`.
- `read()` – reads a number from standard input in the base dictated by the `ibase` variable.
- `scale(expr)` – returns the number of digits after the decimal point in `expr`.
- `sqrt(expr)` – returns the positive square root of `expr`, given that `expr` is non-negative.

Here's a sample execution of the above functions:

```
$ bc -ql
length(000023.450) # Number of significant digits
5
```

```
scale(000023.450) # Number of digits after the decimal
3
sqrt(2) # Square root of 2
1.41421356237309504880
sqrt(-1) # Square root of -1 is an error
Runtime error (func=(main), adr=4): Square root of a negative
number
ibase=2 # Changing the input base to 2
x=read() # Wait to read the input in binary and then display
1100 # This is the input
x # Display the read value in the default output base 10
12
quit # Get out
```

Wait! Are those *all* the built-in functions? What about the previously used `print`? In BC, `print` is not a function; the missing `()` indicates it is actually a statement, like `if`, `for`, ... and its syntax is `print <list>`, where `<list>` is a comma-separated list of strings and expressions. (If you have not yet got the hang of this word *expression*, it is a statement of numbers and variables operated with the various operators and functions.)